

P12 Cloud deployment

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P11.

1. What to build, in one sentence

Make one Node service serve both the API and the built screens from one address, ready to deploy on any always-on host next to Atlas, with a container file, a smoke check, self-hosted fonts and a written runbook, and list the steps Arya must do by hand to bring staging up.

2. Module

Phase 2 hosting, from `docs/BUILD-PLAN.md` section 7. It touches M0 startup and the client build only.

3. Why

`docs/DEPLOYMENT.md` section 1 decided: one service, one https address, for both screens and API.

Today `server.js` serves only the API, and the client is built to `client/dist` with nothing serving it.

The login cookie is `Secure` and only travels to the address

that set it, and CORS allows exactly one origin. One address removes both problems.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P12-cloud-deployment.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P11 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P11's "after" count. If anything fails, stop and tell me.

5. Files to read first

1. `docs/DEPLOYMENT.md` , all of it.
 2. `docs/GO-LIVE.md` section 1, the gates.
 3. `server/server.js` , `server/config/env.js` , `server/config/database.js` .
 4. `server/controllers/healthController.js` and `server/routes/healthRoutes.js` .
 5. `server/scripts/buildIndexes.js` from P01.
 6. `client/index.html` , `client/vite.config.js` , `client/src/main.jsx` and the global CSS file.
 7. The root, server and client `package.json` files, and `.env.example` .
-

6. Part A. Serve the built client from Express

In `createApp()` in `server/server.js`, after the API routes and before `notFound`, only when `config.isProduction`:

1. Serve `client/dist` as static files.
2. Files under `/assets/` have hashed names from Vite. Send them with `Cache-Control: public, max-age=31536000, immutable`.
3. `index.html` is sent with `Cache-Control: no-cache`, so a new deploy is picked up on the next page load.
4. Any `GET` that is not under `/api/` and does not match a file returns `index.html`, so the React router can handle addresses like `/day-close` on a fresh page load.
5. Anything under `/api/` that matches no route still gets the existing JSON 404 from `notFound`. It must never get `index.html`.
6. If `client/dist/index.html` does not exist when the server starts in production, log a fatal message saying "The client has not been built. Run `npm run build`, then start again." and exit with code 1, the same way the index check from P01 refuses to start.

Find the path to `client/dist` from the server file's own location, the way `env.js` finds the repo root, never from the working directory.

Development stays as it is: Vite serves the client on its own port and proxies `/api` to the server.

7. Part B. Fonts served by us

`client/index.html` loads IBM Plex Sans and IBM Plex Mono from Google's servers.

At the cafe, on a 4G backup line, every request to another site is a delay before the screen can draw, and if that site is slow the app looks broken.

1. Add `@fontsource/ibm-plex-sans` and `@fontsource/ibm-plex-mono` to the client, with only the weights already used: Sans 400, 500, 600, and Mono 400, 500, 600, 700.
2. Import them once, in `client/src/main.jsx`.
3. Remove the three Google Fonts lines from `client/index.html`.
4. Check that the font family names used in the CSS and the design tokens still match what the packages register. Fix the names, not the design, if they differ.

8. Part C. Health, security headers, and the release version

1. `GET /api/v1/health` gains `release`, from a new optional environment variable `RELEASE_VERSION`, or null when unset. Add it to `env.js` and `.env.example` with a comment: "Set by the host or the deploy to the git commit being deployed." Additive: existing fields unchanged.
2. Check the existing `helmet()` content security policy works with the built client: start the server with `NODE_ENV=production` locally after a build, load the

app in a browser, and confirm the console shows no blocked script, style, font or image. If anything is blocked, adjust the policy for that one directive only, with a comment saying why. Never switch the policy off.

3. Confirm the response headers include `Strict-Transport-Security`, which `helmet` sets by default. Do not remove it.

9. Part D. A container file, for any host

Add a `Dockerfile` and a `.dockerignore` at the repo root, so the same build runs on any host that accepts a container.

1. A build stage: Node 20, matching `engines` in `package.json`, running `npm ci` and `npm run build`.
2. A run stage: Node 20 slim, production dependencies only, the server and `client/dist`, running as a non-root user, starting with `npm start`.
3. `.dockerignore` leaves out `node_modules`, every `.env` file, `client/dist`, `.git`, test files and docs.
4. No secret in the image. Every environment variable comes from the host at run time.
5. Add a `HEALTHCHECK` that calls `/api/v1/health`.

Index building is a release step, not a start step: the host runs `npm run db:indexes` before switching traffic to a new version. The `Dockerfile` does not run it.

Keep it host-neutral. Do not add files for a particular host.

10. Part E. The smoke check

`server/scripts/smokeCheck.js`, run as `npm run smoke -`
`- --url https://caffeza-staging.example.com`. Add the script to both `package.json` files.

It makes only reading requests and never signs in, so it is safe against production. It checks, and prints one line per check, pass or fail:

1. `GET /api/v1/health` is 200, `database` is `connected`, and `release` is shown.
2. `GET /` returns HTML containing `<div id="root">`.
3. An address like `/day-close` also returns that HTML, so page reloads work.
4. `GET /api/v1/no-such-route` returns JSON 404, not HTML.
5. The first `/assets/` file named in the HTML returns 200 with the long cache header.
6. The response has `Strict-Transport-Security` and a content security policy.
7. When the address starts with `https://`, a request to the same host over `http://` is either refused or redirected to https.
8. `POST /api/v1/auth/login` with no body returns 400, proving the API answers, without signing in.

Exit code 0 when every check passes, 1 otherwise.

11. Part F. The runbook

Update `docs/DEPLOYMENT.md` so a person can deploy without asking anyone:

1. Section 6, every deploy: the exact commands, in order, including `npm run db:indexes` as the release step and `npm run smoke` at the end.
2. A new section, "Deploying with the container": build, the environment variables to set, the release step, and the health check.
3. Section 2: keep the host criteria, and add a short table to fill in once the host is chosen: host name, region, how deploys happen, how the release step runs, where logs are, the fixed outbound address for Atlas.

Do not choose the host. That is Arya's decision, recorded in the decision log.

12. Tests

1. With `NODE_ENV=production` and a built client in a temporary folder, `GET /` returns the HTML with `Cache-Control: no-cache`, an asset returns the immutable cache header, `GET /day-close` returns the HTML, and `GET /api/v1/no-such-route` returns the JSON 404. Point the static path at a temporary folder through a small option on `createApp`, so the test needs no real build.
2. In production with no built client, startup refuses with the fatal message.
3. In development and test, `GET /` is not served by Express, exactly as today.

4. `GET /api/v1/health` includes `release`, null when unset.
5. The smoke check's pass and fail logic for each check, against a local test server.

Run the full suite at the end. Every test that passed before must still pass.

13. Steps Arya does by hand

Print this checklist at the end of your summary, unchanged, so Arya can work through it. These need accounts, payment and judgement, so a coding session must not do them.

1. Choose the host, against `docs/DEPLOYMENT.md` section 2. It must give a fixed outbound address, or stop and decide together.
2. Create the Atlas staging cluster, `docs/DEPLOYMENT.md` section 3, allowing only the host's address.
3. Buy the domain and point `caffeza-staging.<domain>` at the host.
4. Set every environment variable from `docs/DEPLOYMENT.md` section 4 on the host.
`NODE_ENV=production`, `TRUST_PROXY` per the host's documentation, `CLIENT_ORIGIN` exactly the staging address, new secrets from `openssl rand -base64 48`.
5. Deploy. Run `npm run db:indexes` against staging. Start.
6. Run `npm run smoke -- --url https://caffeza-staging.<domain>`. Every line must pass.

7. `npm run provision:restaurant` against staging for Caffeza's owner.
 8. `npm run setup:restaurant` and `npm run import:menu`, dry run first, then `--apply`.
 9. Sign in on a phone and on a laptop. Open a table, fire, bill, print, pay, close the day.
 10. Set up the uptime monitor on `/api/v1/health`.
 11. Record the host choice and these dates in `docs/PROJECT-STATE.md`.
-

14. Non-negotiable rules that apply

"Secrets live in `.env`. `.env` is in `.gitignore`. Never commit a real secret." The container holds none.

"The server runs in the cloud, next to a separate Atlas cluster. Do not add anything that assumes a machine inside the cafe."

"Never create a bill in production to test something." The smoke check only reads.

From `docs/CONVENTIONS.md` section 11: a new environment variable goes into `.env.example` in the same commit.

15. Checks and golden day

No money changes. The golden day acceptance test from P10 must still pass.

16. Docs to update

1. `docs/DEPLOYMENT.md` as in Part F.
 2. `docs/PROJECT-STATE.md` :
 1. The date line.
 2. "Current stage": "Next: P13, the reports spec.
Staging is brought up by hand from the P12 checklist."
 3. Decision log, dated today:

"In production, Express serves the built client from `client/dist` on the same address as the API, with long caching for hashed assets and none for `index.html` . | One address for the cookie and CORS, and new deploys are picked up on the next page load."

"Fonts are served from our own server through `@fontsource` . | A slow outside font server must not delay the till on a 4G line."

"A host-neutral `Dockerfile` is the deploy unit. Index building is a release step, run before traffic moves. | Any host that runs containers works, and indexes are never built under live traffic."
 4. "What changed recently": a P12 entry at the top, and the oldest moved to the archive.
 3. `docs/prompts/README.md` : mark P12 as Done.
-

17. Out of scope

Choosing or configuring a host, buying a domain, creating Atlas clusters. All by hand, from section 13.

Continuous deployment pipelines.

Offline mode, service workers or caching API data in the browser.

Running more than one server instance. The rate limiter keeps its counts in memory, so version 1 runs one instance.

18. Done when

1. `npm test` passes, with before and after counts recorded.
2. `npm run lint` and `npm run build` pass.
3. Locally: `npm run build`, then start with `NODE_ENV=production` and a local database, open `http://localhost:5000`, sign in, and use the app with no console errors and the fonts showing.
4. Locally, if Docker is installed: `docker build .` succeeds. If Docker is not installed, say so in the summary.
5. `npm run smoke -- --url http://localhost:5000` passes every check except check 7, the https redirect, which reports "skipped for http".
6. Every doc in section 16 is updated.
7. Commits on `main`, one line each, for example:
 - `serve the built client in production`
 - `self-host fonts`
 - `add release version to health`
 - `add dockerfile`

`add smoke check`

`update deployment runbook`

8. Push `main`.
9. Print a short summary: commits, files changed, test counts before and after, the local smoke check output, and the section 13 checklist.